Latent Mamba: Adaptive Continuous Reasoning in Constant Memory

Kye
kye@swarms.world

Technical report, November 2026

Abstract

Chain-of-thought reasoning spends extra compute by writing tokens, and in a Transformer every token also grows the key–value cache. We describe *Latent Mamba*, an architecture that lets a Mamba-2 language model reason in continuous latent space for an input-dependent number of steps while its memory stays constant. The model feeds its normalised final hidden state back as its next input through a learned *thought projector* that adds a thinking-mode embedding, and a *halting head* decides when to stop. We observe that in a selective state-space model, depth recurrence with a shared state is the same computation as interleaving self-generated latent inputs along the sequence, which motivates placing thoughts in time rather than in depth. Training follows a Coconut-style curriculum and then a PonderNet objective. Because the recurrent state has a fixed size, the answer can be scored after every thought step in a single batched pass. We give the algorithms, a memory analysis and a reference implementation verified against the exact recurrence. This report specifies and verifies the method; it does not report results from trained models, and we propose an evaluation protocol for obtaining them.

1 Introduction

Language models solve multi-step problems better when they can compute before answering, usually by emitting a chain of thought (CoT) [15]. This has two costs. Natural-language tokens are a narrow channel for intermediate state, and most of them serve fluency rather than reasoning [10]. In a Transformer, every generated token also adds an entry to the key–value (KV) cache, so memory grows with the length of the reasoning.

Latent reasoning moves the intermediate computation into the model's continuous hidden space [16]. Coconut feeds the last hidden state back as the next input embedding instead of decoding it [10]. Recurrent-depth models iterate a shared block a variable number of times per token [5]. Both let a model think without writing text, but in a Transformer each continuous thought still occupies a cache slot.

Selective state-space models (SSMs) such as Mamba and Mamba-2 [9, 3] carry context in a fixed-size recurrent state instead of a cache, so a thought step costs constant memory: it updates the state and nothing else. We develop this into a concrete architecture and training procedure.

Contributions.

- A *time–depth collapse* for SSMs (Proposition 1): depth recurrence with a shared state equals self-generated inputs interleaved along the sequence. Placing thoughts in time avoids the missing-state problem that per-iteration states create under adaptive exit.
- **Latent Mamba** (Figure 2): a thought projector with a thinking-mode embedding, a halting head, and functional fixed-size state handling. The added parameters are about 0.3% of a 2.7B model.
- A training recipe (Algorithms 1 and 2) and an adaptive inference procedure (Algorithm 3). PonderNet scoring is batched across thought steps, which is cheap only because SSM state is fixed-size.
- A single-file reference implementation, a numerical verification of its kernels (Table 2), and an evaluation protocol with falsifiable hypotheses (Section 6).

2 Background

Mamba-2. Each layer projects its input u_t to a gate z_t, a branch (x_t, b_t, c_t) that passes through a causal depthwise convolution of width K, and per-head step sizes $\Delta_t = \mathrm{softplus}(w^\top u_t + \beta) > 0$. For head h with decay rate $a_h < 0$, head input $x_t \in \mathbb{R}^P$ and

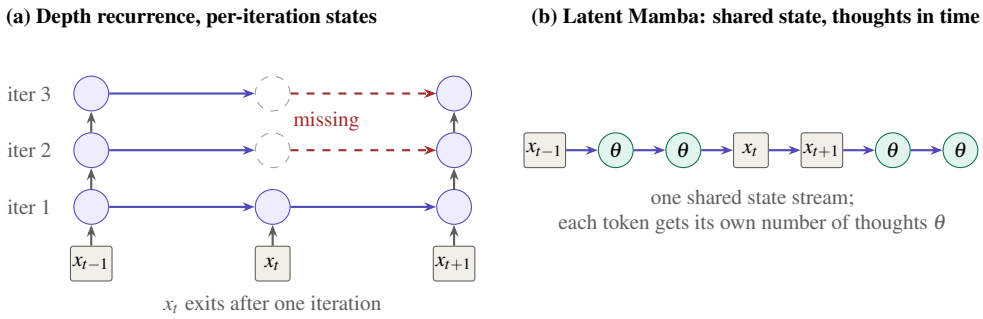


Figure 1: (a) Per-iteration state streams break under adaptive exit: when x_t halts after one iteration, the deeper iterations at x_{t+1} have no state to read. (b) With a shared state, depth recurrence becomes thoughts interleaved in time (Proposition 1). Purple arrows carry SSM state.

$b_t, c_t \in \mathbb{R}^N$,

$$S_t = e^{\Delta_t a_h} S_{t-1} + \Delta_t x_t b_t^\top, \qquad y_t = S_t c_t + D_h x_t, \tag{1}$$

followed by a gated RMSNorm and an output projection [3]. The recurrent state of a layer is the convolution window (the last $K-1$ inputs) together with $S \in \mathbb{R}^{H\times P\times N}$. Training processes whole sequences with the chunked SSD algorithm; inference applies Equation (1) one token at a time. Both compute the same function.

Continuous thought. Coconut [10] marks a latent span with `<bot>` and `<eot>`. Inside the span the final hidden state is fed back as the next input embedding. It is trained with a curriculum adapted from implicit CoT [4]: at stage k, the first k language reasoning steps are replaced by ck continuous thoughts, and the loss covers only the remaining text.

Adaptive computation. ACT [7] and PonderNet [2] let a network choose how many steps to run. PonderNet predicts, at each step n, a conditional halting probability λ_n. This gives the distribution $p_n = \lambda_n \prod_{j<n}(1-\lambda_j)$. Training minimises $\sum_n p_n \mathscr{L}_n + \beta\,\mathrm{KL}(p \,\|\, \mathrm{Geo}(\lambda_p))$, where $\mathscr{L}_n$ is the task loss after step n.

3 Latent Mamba

3.1 Where should a recurrent model think?

A model can add computation along two axes [16, 1]: *depth* (iterate a block several times at one position) and *time* (insert extra positions). Recurrent-depth Transformers take the depth route [5]. In an SSM the depth route raises a question that attention does not: which recurrent state does iteration i at position t read?

If each iteration has its own state stream, adaptive exit breaks the streams (Figure 1a). When token t stops after one iteration, iterations two and up at token $t+1$ have no state from position t to read. If instead all iterations share one state, the problem disappears and the computation becomes recurrence in time.

Proposition 1 (Time–depth collapse)**.** *Let $F : (u, S) \mapsto (h, S')$ be one pass of an SSM stack over a single input, and let g map outputs to inputs. Suppose the model runs r_t iterations at position t. Iteration $i+1$ takes input $g(h_t^{(i)})$ and the state $S_t^{(i)}$ left by iteration i; iteration 1 at position $t+1$ takes u_{t+1} and $S_t^{(r_t)}$. Then every output and state equals that of a single pass of F over the interleaved sequence $\big(u_1, g(h_1^{(1)}), \ldots, g(h_1^{(r_1-1)}), u_2, \ldots\big)$.*

Proof sketch. Both procedures apply F to the same (input, state) pairs in the same order. Induct on the position in the interleaved sequence. □

The statement is elementary, but it fixes the design. In an SSM, shared-state depth recurrence *is* thinking in time. That form keeps memory constant, needs no new kernels, and has no missing-state problem, so Latent Mamba places its thoughts along the sequence (Figure 1b). The feedback map g is the thought projector.

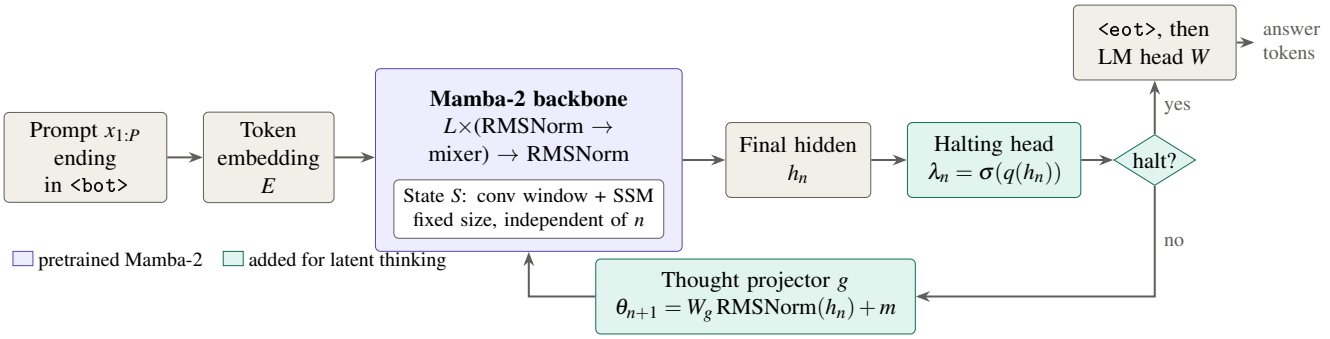


Figure 2: Latent Mamba. The prompt is prefilled with the parallel scan. Each thought feeds the projected final hidden state back as the next input and updates only the fixed-size state S. The halting head decides between another thought and decoding the answer after <eot>.

3.2 Architecture

The prompt ends in <bot>. Let h_0 be the final normalised hidden state at <bot>, S_0 the state after the prompt, and F one recurrent step of the whole backbone. Thought step $n \geq 1$ computes

$$\theta_n = W_g\,\mathrm{RMSNorm}_\gamma(h_{n-1}) + m, \tag{2}$$

$$(h_n, S_n) = F(\theta_n, S_{n-1}), \qquad \lambda_n = \sigma\big(q(h_n)\big). \tag{3}$$

After halting, the model reads <eot> and decodes the answer with the language-model head W. Table 1 lists the added parameters.

Thought projector. The final hidden state lives in output space, not input-embedding space. Mamba computes Δ_t, b_t and c_t from its input, so off-distribution inputs push these projections into regimes never seen in training. We therefore initialise $W_g = I$, $m = 0$ and γ to the RMS of the embedding matrix. At initialisation, the model is exactly Coconut with thoughts at the scale of a token embedding.

Thinking-mode embedding and prompt retention. By Equation (1), after n thoughts the part of head h's state that came from the prompt has been multiplied by

$$\textstyle\prod_{k=1}^{n} e^{\Delta_{h,k} a_h} \;=\; \exp\big(a_h \sum_{k=1}^{n} \Delta_{h,k}\big). \tag{4}$$

Long thinking can thus erase the question. $\Delta_{h,k}$ is a function of the input, so the learned vector m gives the model a direct way to select small step sizes while thinking. This retains the prompt, while thought content still enters through the convolution and the x and b paths. The cumulative step size $\sum_k \Delta_{h,k}$ is a cheap diagnostic of prompt decay.

Halting head. q is a two-layer MLP. Its last layer has zero weights and bias $\mathrm{logit}(\lambda_p)$, so the initial halting distribution equals the geometric prior and the initial KL term is zero.

Why thoughts add expressivity. Within one pass, each layer's state update in Equation (1) is linear in S. A thought composes the full nonlinear network, including gates, normalisation and all layers, before the next state update. So n thoughts form a nonlinear recurrence of depth $n \cdot L$ in latent space, applied to whatever the state already holds.

3.3 Memory

The state of the whole model occupies

$$L\left[(K-1)(d_{\text{inner}} + 2N) + HPN\right] \tag{5}$$

values, independent of both prompt length and thought count. For the Mamba-2 2.7B configuration (L=64, d=2560, N=128, P=64, K=4) this is 671,488 values per layer, or 82.0 MiB in fp16. A 32-layer Transformer of the same width with multi-head attention stores 0.3125 MiB of KV cache per token; with 4$\times$ grouped-query attention, 0.078 MiB.

Figure 3 compares the three after a 512-token prompt. The grouped-query Transformer is smaller until about 1,050 cached tokens, so the advantage is not universal. Beyond that point every Transformer thought costs memory, while Latent Mamba's footprint stays flat at any thinking budget.

Table 1: Parameters added to a Mamba-2 2.7B model ($d = 2560$).

Module	Params	Share
Thought projector (d^2+2d)	6.56M	0.24%
Halting head ($\frac{d^2}{4}+\frac{d}{2}+1$)	1.64M	0.06%
Total	8.20M	0.30%

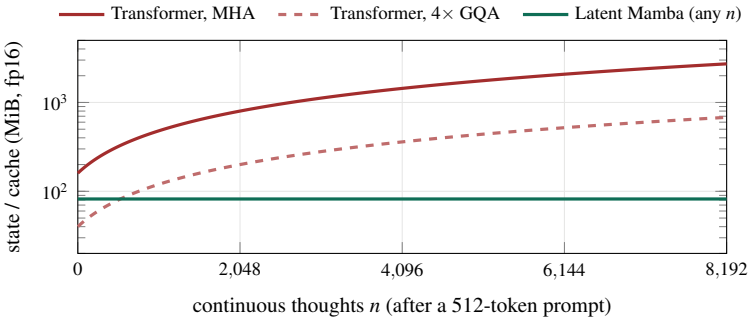


Figure 3: Analytic memory for a 2.7B-scale configuration at batch size 1. Transformer KV cache grows with every thought; the Latent Mamba state does not.

Algorithm 1 Curriculum loss with per-example thought counts

Require: prompts $x_{1:P}$ ending in `<bot>`; targets $y_{1:T}$; counts n_b

$(H, S) \leftarrow \textsc{Backbone}(E(x), \mathbf{0})$ ▷ chunked scan
$h \leftarrow H_P$ ▷ hidden state at `<bot>`
for $i = 1, \ldots, \max_b n_b$ **do**
  $(h', S') \leftarrow F\big(W_g \text{RMSNorm}(h) + m,\ S\big)$ ▷ one step
  $a_b \leftarrow [\, i \le n_b \,]$ for every example b
  $h \leftarrow a \odot h' + (1-a) \odot h;\ \ S \leftarrow a \odot S' + (1-a) \odot S$
$Z \leftarrow \textsc{Backbone}\big(E([\texttt{<eot>}, y_{1:T-1}]), S\big)$ ▷ teacher forcing
return $\text{CE}(WZ, y)$ over non-padding targets

4 Training and inference

4.1 Curriculum

We follow Coconut [10] (Algorithm 1). Stage 0 is language CoT after `<bot><eot>`, i.e. zero thoughts. At stage k, the first $\min(k, s)$ of an example's s reasoning steps are replaced by $c\min(k, s)$ thoughts, and the target keeps only the remaining steps and the answer. Thought counts differ within a batch. An example that has finished thinking is frozen by selecting its old state and hidden vector, so one batch runs $\max_b n_b$ steps. Gradients flow through every thought. For long budgets, the state can be detached after a fixed number of steps, as in recurrent-depth training [5]. We reset the optimiser at each stage, as Coconut does.

4.2 Learning to halt

The final stage trains the halting head with the PonderNet objective (Algorithm 2). The model takes N thoughts and keeps the state after each. Because these states have a fixed size, the N branches can be concatenated along the batch axis, and the answer is scored from all of them in one pass of $N{\cdot}B$ short sequences. The extra cost is about N answer passes. In a Transformer, the same branching would copy N KV caches of growing length.

We set $\lambda_N{=}1$ so that p is a proper distribution, and use the geometric prior truncated to $\{1, \ldots, N\}$. Two cheaper alternatives need no halting training: a zero-shot convergence rule, as used for recurrent-depth models [5, 13], and reinforcement learning with a per-thought compute penalty.

4.3 Adaptive inference

Algorithm 3 thinks until a halting rule fires and then decodes. The learned rule stops at the median of the predicted halting distribution, when $1 - \prod_j (1 - \lambda_j) \ge \tau$ with $\tau = 0.5$. The convergence rule stops when $\|h_n - h_{n-1}\| / \|h_{n-1}\| < \varepsilon$. A fixed rule runs exactly N thoughts, matching the curriculum stages. Within a batch, halted examples are frozen while the others continue, and memory is constant throughout.

5 Reference implementation

We provide a single-file PyTorch implementation with no custom kernels. Sequences use the chunked SSD algorithm [3] started from an arbitrary initial state; single tokens use the exact recurrence of Equation (1). State updates are functional: they

Algorithm 2 PonderNet step with batched branch scoring

Require: prompts x ending in `<bot>`; targets y; budget N; prior λ_p; weight β

$(H, S) \leftarrow \textsc{Backbone}(E(x), \mathbf{0})$; $h \leftarrow H_P$
for $n = 1, \ldots, N$ **do**
  $(h, S) \leftarrow F\big(W_g \mathrm{RMSNorm}(h) + m,\ S\big)$
  $\lambda_n \leftarrow \sigma(q(h))$; $S^{(n)} \leftarrow S$
$\lambda_N \leftarrow 1$; $p_n \leftarrow \lambda_n \prod_{j<n}(1-\lambda_j)$ ▷ log space
$\bar{S} \leftarrow \textsc{Concat}_{\text{batch}}(S^{(1)}, \ldots, S^{(N)})$
$\ell_{1:N} \leftarrow$ per-example CE of y decoded from $\bar{S}$ ▷ one pass
$g_n \propto \lambda_p(1-\lambda_p)^{n-1}$, normalised on $\{1..N\}$
return $\sum_n p_n \ell_n + \beta \sum_n p_n \log(p_n / g_n)$

Algorithm 3 Adaptive latent generation

Require: prompt x ending in `<bot>`; maximum N; threshold τ

$(H, S) \leftarrow \textsc{Backbone}(E(x), \mathbf{0})$; $h \leftarrow H_P$
$\rho \leftarrow 1$; $n \leftarrow 0$ ▷ ρ: probability of not halting
while $n < N$ **do**
  $(h, S) \leftarrow F\big(W_g \mathrm{RMSNorm}(h) + m,\ S\big)$; $n \leftarrow n + 1$
  $\rho \leftarrow \rho\,(1 - \sigma(q(h)))$
  if $1 - \rho \geq \tau$ **then break**
$v \leftarrow$ `<eot>`
repeat
  $(h, S) \leftarrow F(E(v), S)$; $v \leftarrow \arg\max Wh$; emit v
until end token or length limit

Table 2: Numerical checks of the kernels, NumPy port (float64; batch 3, length 21, H=4, P=8, N=16, chunk 8).

Check	Max abs. error
Chunked scan vs. recurrence, outputs	1.1×10^{-14}
Chunked scan vs. recurrence, final state	2.7×10^{-15}
Prefill + per-token continuation vs. one pass	2.2×10^{-15}
final state after continuation	6.7×10^{-16}
Halting distribution sums to one	machine precision

return new tensors, so a cache can be branched, frozen per example or concatenated without side effects, as Algorithms 1 and 2 require. The mixer follows Mamba-2's parameterisation, so pretrained weights can be loaded by renaming keys. The file also contains a diagnostic ablation that replaces thought content with zeros, which turns thoughts into pause tokens [6], or with noise of matching RMS.

Verification. The method depends on one property: continuing from a stored state must give exactly the result of an uninterrupted pass. We ported the kernels line for line to NumPy in float64 and tested this property directly (Table 2). The scan used three chunks with padding and a random initial state. The file's built-in self-test repeats these checks in PyTorch and adds gradient and generation tests.

6 Proposed evaluation

We have not yet trained Latent Mamba, so we state the protocol and the predictions it tests.

Probe task. k-hop pointer chasing: the prompt lists a fresh random permutation f over S symbols as pairs, then a start symbol x and a hop count k. The answer is $f^k(x)$, and the CoT is $f(x), \ldots, f^{k-1}(x)$. Difficulty is set by k alone, which makes adaptive allocation easy to read.

Measurements and ablations. Accuracy and mean thoughts by k; accuracy against a fixed thought budget; peak memory and latency against a Coconut Transformer of matched size. Ablations: no thoughts; zero and noise thought content; no mode

embedding; learned versus convergence halting.

Predictions. **H1**: For $k>1$, accuracy falls to near chance without thoughts and under zero ablation, but not for $k=1$. **H2**: Learned halting spends more thoughts on larger k. **H3**: Without the mode embedding, accuracy degrades at long budgets, and the cumulative step size in Equation (4) grows. Beyond the probe, we would start from a pretrained Mamba-2 checkpoint and follow Coconut's GSM8K and ProsQA settings.

7 Related work

Latent reasoning. Coconut [10] introduced continuous thoughts and observed breadth-first-search-like behaviour. Implicit CoT [4] internalises reasoning steps through a curriculum. Pause tokens [6] add compute without content. Zhu et al. [16] survey the area and distinguish vertical from horizontal recurrence.

Recurrent depth. Geiping et al. [5] train a depth-recurrent Transformer with randomly sampled iteration counts and obtain zero-shot per-token adaptive exit. RD-VLA [13] stops when the latent state converges. SST V2 [1] streams a latent state across positions in a Transformer and deliberates per position. Wang and Reid [14] place Mamba-2 hybrid blocks inside a tiny recursive reasoning model [11]. They leave open whether the recursion can be folded into the SSM's own state updates; Proposition 1 and Latent Mamba give one such folding.

Adaptive computation and SSM expressivity. We use ACT [7] and PonderNet [2] directly. Diagonal SSMs with positive decay cannot track some states in a single pass [12]; allowing negative eigenvalues helps [8].

8 Limitations

No trained-model results: this report specifies and verifies the method, and the predictions of Section 6 are untested. **Sequential training:** each thought depends on the previous output, so thoughts cannot use the parallel scan and training cost grows linearly with the budget; parallel training of nonlinear recurrences [1] may help. **Fixed-size memory cuts both ways:** thinking cannot restore prompt details the state has already lost, so recall-heavy tasks may need hybrid attention layers, with thoughts kept out of their KV cache. **Interpretability:** thoughts are not text, and decoding them through W is only a probe. **Halting collapse:** the head can learn to always stop early, so β and λ_p need tuning.

9 Conclusion

In a selective SSM, extra depth with a shared state is the same thing as extra time. Latent Mamba uses this to let a Mamba-2 model think in continuous space for as long as each input requires, at constant memory and with about 0.3% more parameters. The next step is the evaluation in Section 6.

References

[1] T. Aviss. State Stream Transformer (SST) V2: Parallel training of nonlinear recurrence for latent space reasoning. arXiv:2605.00206, 2026.
[2] A. Banino, J. Balaguer, C. Blundell. PonderNet: Learning to ponder. arXiv:2107.05407, 2021.
[3] T. Dao, A. Gu. Transformers are SSMs: Generalized models and efficient algorithms through structured state space duality. ICML, 2024.
[4] Y. Deng, Y. Choi, S. Shieber. From explicit CoT to implicit CoT: Learning to internalize CoT step by step. arXiv:2405.14838, 2024.
[5] J. Geiping, S. McLeish, N. Jain, et al. Scaling up test-time compute with latent reasoning: A recurrent depth approach. NeurIPS, 2025.
[6] S. Goyal, Z. Ji, A. S. Rawat, et al. Think before you speak: Training language models with pause tokens. ICLR, 2024.
[7] A. Graves. Adaptive computation time for recurrent neural networks. arXiv:1603.08983, 2016.
[8] R. Grazzi, J. Siems, A. Zela, et al. Unlocking state-tracking in linear RNNs through negative eigenvalues. ICLR, 2025.
[9] A. Gu, T. Dao. Mamba: Linear-time sequence modeling with selective state spaces. arXiv:2312.00752, 2023.
[10] S. Hao, S. Sukhbaatar, D. Su, et al. Training large language models to reason in a continuous latent space. arXiv:2412.06769, 2024.
[11] A. Jolicoeur-Martineau. Less is more: Recursive reasoning with tiny networks. arXiv preprint, 2025.
[12] W. Merrill, J. Petty, A. Sabharwal. The illusion of state in state-space models. ICML, 2024.
[13] Y. Tur et al. Recurrent-Depth VLA: Implicit test-time compute scaling of VLA models via latent iterative reasoning. arXiv:2602.07845, 2026.
[14] W. Wang, F. Reid. Tiny recursive reasoning with Mamba-2 attention hybrid. arXiv:2602.12078, 2026.
[15] J. Wei, X. Wang, D. Schuurmans, et al. Chain-of-thought prompting elicits reasoning in large language models. NeurIPS, 2022.
[16] R.-J. Zhu et al. A survey on latent reasoning. arXiv:2507.06203, 2025.